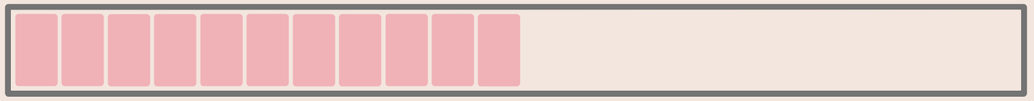
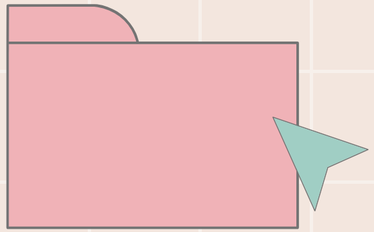
RESTAURANT ORDERING SYSTEM

TEAM G:
RAKI WANE, ZAINA OTABASHI





RAKI WANE - PRODUCT OWNER

ZAINA OTABASHI - SCRUM MASTER

INTRODUCTION

WE BUILT A REST API THAT ENABLES ONLINE FOOD ORDERING FOR CUSTOMERS AND STREAMLINES OPERATIONS FOR RESTAURANT STAFF.

- ENABLE CUSTOMERS TO ORDER FOOD ONLINE WITHOUT CREATING AN ACCOUNT
- HELP STAFF MANAGE MENU ITEMS, ORDERS, INGREDIENTS, AND PROMOTIONS
- PROVIDE INSIGHT INTO SALES, CUSTOMER REVIEWS, AND DISH PERFORMANCE

KEY FEATURES & OBJECTIVES

OBJECTIVES

- Build an API using Python (FastAPI)
- Integrate with a MySQL database
- Implement full CRUD functionality

KEY FEATURES

- FastAPI-based backend
- Unit testing using pytest
- Structured database with clearly defined relationships and primary keys

KEY FEATURES

- Full CRUD functionality implemented across all database tables
- API endpoints that address both customer and restaurant staff requirements

DEMO

FAST API DEMO

http://127.0.0.1:8000/docs

For this demo, we will first retrieve orders, by executing GET/orders/

Send

Next, we will create a new order using POST/orders/.

We can use: {

"customer_id": 1,

"tracking_number": "TRACK999",

"status": "Preparing",

"total_price": 10.50

}

To check correct implementation, we can run GET/orders/ again and check if the order was added.

Finally, we can delete the order using DELETE/orders/{id}.

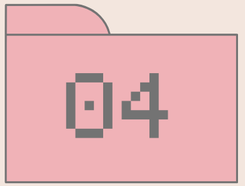
We can run GET/orders/ again to ensure that it worked.

KEY CHALLENGES & LEARNING

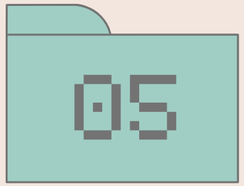
DIVIDING TEAMWORK WHILE KEEPING CODEBASE CONSISTANT

GETTING DATABASE SET UP

STRUCTURING REST API FROM MODELS TO ENDPOINTS



USING SWAGGER UI TO TEST AND DEMO API'S



LEARNING GIT/GITHUB WHILE BUILDING PROJECT